Day 11: Streams, Partitioned Logs & Consumer Groups

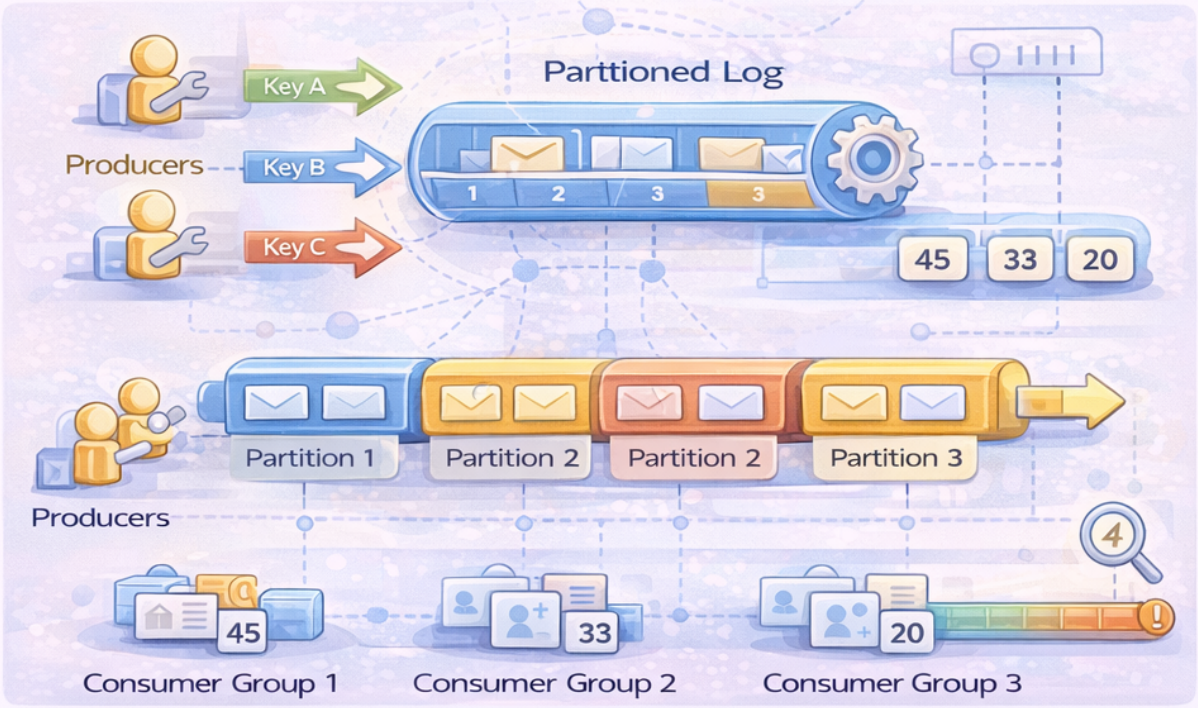
Moving high-throughput event data safely from producers to multiple independent consumers.

Append-Only Logs

- Topic stores a history of events
- Events are written, but never updated in place
- Replay and auditing made easy.

Partitioning & Ordering

- Topic split across partitions for scale
- Ordering guaranteed within one partition
- Keys route related events to same partition



Consumer Groups & Offsets

- Multiple apps can read the same stream independently
- Each group tracks its own offsets
- Restart from last committed offset

Replay, Lag & Backpressure

- Consumers can replay from old offsets
- Lag shows how behind each group is
- Lag rising = consumers falling behind

Partitions boost throughput, but offset tracking and lag monitoring keep processing reliable.

Day 11: Streams, Partitioned Logs, and Consumer Groups

Day 11 focuses on how event-driven systems move high-throughput data safely from producers to many independent consumers.

It covers:

• append-only event logs

• partitioning for scale

• ordering guarantees and key-based routing

• consumer groups and offset tracking

• replay and recovery

• lag monitoring and backpressure

In short, Day 11 is about building reliable stream processing foundations where data can be consumed, retried, and replayed without tight coupling.

Core ideas

Append-only logs

• A `stream log` stores events in order and never rewrites historical records.

• Producers append events, while consumers read by offset.

• This model makes replay and auditing straightforward.

Partitioning and ordering

• A topic is split into `partitions` so writes and reads can scale horizontally.

• Ordering is guaranteed within one partition, not across all partitions.

• Key-based partitioning keeps related events together (for example, all events for one user).

Consumer groups and offsets

• Each consumer group tracks its own committed offset per partition.

• Different groups can process the same stream independently.

• If a group crashes, it can resume from committed offsets instead of starting over.

Replay, lag, and backpressure

• Replaying from older offsets helps recovery, reprocessing, and debugging.

• `Consumer lag` shows how far a group is behind the latest offset.

• Growing lag signals that consumers are slow or under-provisioned.

Day-11 sample: Partitioned Log with Consumer Offsets

This repository includes a small partitioned log sample in both Python and Java.

Functional requirements

• Produce events with key-based partition routing

• Register multiple consumer groups

• Poll events without auto-committing offsets

• Commit processed offsets explicitly

• Compute per-partition lag for each consumer group

High-level components

• `Producer`: appends keyed events

• `PartitionedLog`: stores events by partition and offset

• `Consumer group state`: tracks committed offsets per partition

• `Poll logic`: reads events from committed offsets

• `Commit logic`: advances offsets after successful processing

Data flow

1. Producer appends events into a partitioned log
2. A consumer group polls a batch from its current committed offsets
3. Successfully processed events are committed
4. Uncommitted events are replayed on the next poll
5. Lag reflects how far each group is behind

Project structure

```
Day-11/
  README.md
  partitioned-log-sample/
    python/
      partitioned_log.py
      demo.py
    java/
      PartitionedLog.java
      Main.java
```

Run the sample

Python

```
cd partitioned-log-sample\python
python demo.py
```

Java

```
cd partitioned-log-sample\java
javac Main.java PartitionedLog.java
java Main
```

Interview takeaways

• Partitioning increases throughput but limits ordering guarantees to each partition

- Consumer groups isolate processing concerns while sharing the same source stream
- Explicit offset commits are central to recovery and duplicate handling
- Replay is a feature, not an accident, in stream systems
- Lag is one of the most important operational indicators in event-driven architectures

Next improvements

- Add retention windows and offset expiration
- Add consumer rebalancing simulation across multiple members
- Add idempotency keys for downstream consumers
- Add delayed retries and poison-event routing
- Replace in-memory log with Kafka, Pulsar, or Kinesis

Source Code Appendix

Day-11/partitioned-log-sample/python/partitioned_log.py

```
from __future__ import annotations

import hashlib
from dataclasses import dataclass
from typing import Dict, Generic, TypeVar

T = TypeVar("T")


@dataclass(frozen=True)
class Message(Generic[T]):
    partition: int
    offset: int
    key: str
    payload: T


class PartitionedLog(Generic[T]):
    def __init__(self, topic: str, partitions: int) -> None:
        if not topic:
            raise ValueError("topic must be non-empty")
        if partitions <= 0:
            raise ValueError("partitions must be positive")

        self.topic = topic
        self._partitions: list[list[Message[T]]] = [[] for _ in range(partitions)]
        self._committed_offsets: Dict[str, list[int]] = {}

    def produce(self, key: str, payload: T) -> Message[T]:
        partition_id = self._partition_for_key(key)
        offset = len(self._partitions[partition_id])
        message = Message(partition=partition_id, offset=offset, key=key, payload=payload)
        self._partitions[partition_id].append(message)
        return message

    def register_group(self, group_name: str) -> None:
        if group_name not in self._committed_offsets:
            self._committed_offsets[group_name] = [0] * len(self._partitions)

    def poll(self, group_name: str, max_messages: int = 1) -> list[Message[T]]:
        if max_messages <= 0:
            raise ValueError("max_messages must be positive")

        committed = self._offsets_for_group(group_name)
        cursors = committed.copy()
        batch: list[Message[T]] = []

        while len(batch) < max_messages:
            emitted = False
            for partition_id, entries in enumerate(self._partitions):
                cursor = cursors[partition_id]
                if cursor >= len(entries):
                    continue

                batch.append(entries[cursor])
                cursors[partition_id] += 1
                emitted = True

                if len(batch) >= max_messages:
                    break

            if not emitted:
                break

        return batch
```

```
    def commit(self, group_name: str, messages: list[Message[T]]) -> None:
        committed = self._offsets_for_group(group_name)
        for message in messages:
            next_offset = message.offset + 1
            if next_offset > committed[message.partition]:
                committed[message.partition] = next_offset

    def lag(self, group_name: str) -> dict[int, int]:
        committed = self._offsets_for_group(group_name)
        return {
            partition_id: len(entries) - committed[partition_id]
            for partition_id, entries in enumerate(self._partitions)
        }

    def committed_offsets(self, group_name: str) -> dict[int, int]:
        committed = self._offsets_for_group(group_name)
        return {partition_id: offset for partition_id, offset in enumerate(committed)}

    def _offsets_for_group(self, group_name: str) -> list[int]:
        offsets = self._committed_offsets.get(group_name)
        if offsets is None:
            raise KeyError(f"unknown consumer group: {group_name}")
        return offsets

    def _partition_for_key(self, key: str) -> int:
        digest = hashlib.md5(key.encode("utf-8")).hexdigest()
        return int(digest, 16) % len(self._partitions)
```

Day-11/partitioned-log-sample/python/demo.py

```
from partitioned_log import Message, PartitionedLog


def print_batch(label: str, batch: list[Message[str]]) -> None:
    print(label)
    if not batch:
        print("  (empty)")
        return

    for message in batch:
        print(
            "  "
            + f"p{message.partition}@{message.offset} "
            + f"key={message.key} payload={message.payload}"
        )


def main() -> None:
    log = PartitionedLog[str](topic="orders", partitions=3)

    events = [
        ("user-1", "order-1001-created"),
        ("user-2", "order-2001-created"),
        ("user-1", "order-1001-paid"),
        ("user-3", "order-3001-created"),
        ("user-2", "order-2001-cancelled"),
    ]

    for key, payload in events:
        message = log.produce(key=key, payload=payload)
        print(
            "produce -> "
            + f"p{message.partition}@{message.offset} "
            + f"key={message.key} payload={message.payload}"
        )

    log.register_group("billing")
    log.register_group("analytics")

    print("billing lag before poll:", log.lag("billing"))
```

```
    billing_first = log.poll("billing", max_messages=4)
    print_batch("billing poll #1 (before commit)", billing_first)
    print("billing committed offsets:", log.committed_offsets("billing"))

    log.commit("billing", billing_first[:2])
    print("billing committed after partial commit:", log.committed_offsets("billing"))
    print("billing lag after partial commit:", log.lag("billing"))

    billing_second = log.poll("billing", max_messages=4)
    print_batch("billing poll #2 (uncommitted messages replay)", billing_second)

    log.commit("billing", billing_second)
    print("billing committed after full commit:", log.committed_offsets("billing"))
    print("billing lag final:", log.lag("billing"))

    analytics_first = log.poll("analytics", max_messages=3)
    print_batch("analytics poll #1 (independent group)", analytics_first)
    print("analytics committed offsets:", log.committed_offsets("analytics"))


if __name__ == "__main__":
    main()
```

Day-11/partitioned-log-sample/java/PartitionedLog.java

```
import java.util.ArrayList;
import java.util.Arrays;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;
import java.math.BigInteger;
import java.nio.charset.StandardCharsets;
import java.security.MessageDigest;
import java.security.NoSuchAlgorithmException;

public class PartitionedLog<T> {
    public static final class Message<T> {
        private final int partition;
        private final int offset;
        private final String key;
        private final T payload;

        private Message(int partition, int offset, String key, T payload) {
            this.partition = partition;
            this.offset = offset;
            this.key = key;
            this.payload = payload;
        }

        public int getPartition() {
            return partition;
        }

        public int getOffset() {
            return offset;
        }

        public String getKey() {
            return key;
        }

        public T getPayload() {
            return payload;
        }
    }

    private final String topic;
    private final List<List<Message<T>>> partitions;
    private final Map<String, int[]> committedOffsets;
    public PartitionedLog(String topic, int partitionCount) {
```

```
        if (topic == null || topic.isBlank()) {
            throw new IllegalArgumentException("topic must be non-empty");
        }
        if (partitionCount <= 0) {
            throw new IllegalArgumentException("partitionCount must be positive");
        }

        this.topic = topic;
        this.partitions = new ArrayList<>();
        for (int i = 0; i < partitionCount; i += 1) {
            this.partitions.add(new ArrayList<>());
        }
        this.committedOffsets = new LinkedHashMap<>();
    }

    public String getTopic() {
        return topic;
    }

    public Message<T> produce(String key, T payload) {
        int partitionId = partitionForKey(key);
        List<Message<T>> partition = partitions.get(partitionId);
        int offset = partition.size();

        Message<T> message = new Message<>(partitionId, offset, key, payload);
        partition.add(message);
        return message;
    }

    public void registerGroup(String groupName) {
        committedOffsets.computeIfAbsent(groupName, ignored -> new int[partitions.size()]);
    }

    public List<Message<T>> poll(String groupName, int maxMessages) {
        if (maxMessages <= 0) {
            throw new IllegalArgumentException("maxMessages must be positive");
        }

        int[] committed = offsetsForGroup(groupName);
        int[] cursors = Arrays.copyOf(committed, committed.length);
        List<Message<T>> batch = new ArrayList<>();

        while (batch.size() < maxMessages) {
            boolean emitted = false;

            for (int partitionId = 0; partitionId < partitions.size(); partitionId += 1) {
                List<Message<T>> entries = partitions.get(partitionId);
                int cursor = cursors[partitionId];
                if (cursor >= entries.size()) {
                    continue;
                }

                batch.add(entries.get(cursor));
                cursors[partitionId] += 1;
                emitted = true;

                if (batch.size() >= maxMessages) {
                    break;
                }
            }

            if (!emitted) {
                break;
            }
        }

        return batch;
    }

    public void commit(String groupName, List<Message<T>> messages) {
```

```
        int[] committed = offsetsForGroup(groupName);
        for (Message<T> message : messages) {
            int nextOffset = message.getOffset() + 1;
            int partitionId = message.getPartition();
            if (nextOffset > committed[partitionId]) {
                committed[partitionId] = nextOffset;
            }
        }
    }

    public Map<Integer, Integer> lag(String groupName) {
        int[] committed = offsetsForGroup(groupName);
        Map<Integer, Integer> lag = new LinkedHashMap<>();

        for (int partitionId = 0; partitionId < partitions.size(); partitionId += 1) {
            int partitionSize = partitions.get(partitionId).size();
            lag.put(partitionId, partitionSize - committed[partitionId]);
        }

        return lag;
    }

    public Map<Integer, Integer> committedOffsets(String groupName) {
        int[] committed = offsetsForGroup(groupName);
        Map<Integer, Integer> offsets = new LinkedHashMap<>();
        for (int partitionId = 0; partitionId < committed.length; partitionId += 1) {
            offsets.put(partitionId, committed[partitionId]);
        }
        return offsets;
    }

    private int[] offsetsForGroup(String groupName) {
        int[] offsets = committedOffsets.get(groupName);
        if (offsets == null) {
            throw new IllegalArgumentException("unknown consumer group: " + groupName);
        }
        return offsets;
    }

    private int partitionForKey(String key) {
        try {
            byte[] digest = MessageDigest.getInstance("MD5").digest(key.getBytes(StandardCharsets.UT
            BigInteger value = new BigInteger(1, digest);
            return value.mod(BigInteger.valueOf(partitions.size())).intValue();
        } catch (NoSuchAlgorithmException e) {
            throw new IllegalStateException("MD5 algorithm unavailable", e);
        }
    }
}
```

Day-11/partitioned-log-sample/java/Main.java

```
import java.util.List;

public class Main {
    public static void main(String[] args) {
        PartitionedLog<String> log = new PartitionedLog<>("orders", 3);

        String[][] events = {
            {"user-1", "order-1001-created"},
            {"user-2", "order-2001-created"},
            {"user-1", "order-1001-paid"},
            {"user-3", "order-3001-created"},
            {"user-2", "order-2001-cancelled"}
        };

        for (String[] event : events) {
            PartitionedLog.Message<String> message = log.produce(event[0], event[1]);
            System.out.println(
                "produce -> p"
```

```
                    + message.getPartition()
                    + "@"
                    + message.getOffset()
                    + " key="
                    + message.getKey()
                    + " payload="
                    + message.getPayload()
            );
        }

        log.registerGroup("billing");
        log.registerGroup("analytics");

        System.out.println("billing lag before poll: " + log.lag("billing"));

        List<PartitionedLog.Message<String>> billingFirst = log.poll("billing", 4);
        printBatch("billing poll #1 (before commit)", billingFirst);
        System.out.println("billing committed offsets: " + log.committedOffsets("billing"));

        log.commit("billing", billingFirst.subList(0, Math.min(2, billingFirst.size())));
        System.out.println("billing committed after partial commit: " + log.committedOffsets("billin
        System.out.println("billing lag after partial commit: " + log.lag("billing"));

        List<PartitionedLog.Message<String>> billingSecond = log.poll("billing", 4);
        printBatch("billing poll #2 (uncommitted messages replay)", billingSecond);

        log.commit("billing", billingSecond);
        System.out.println("billing committed after full commit: " + log.committedOffsets("billing")
        System.out.println("billing lag final: " + log.lag("billing"));

        List<PartitionedLog.Message<String>> analyticsFirst = log.poll("analytics", 3);
        printBatch("analytics poll #1 (independent group)", analyticsFirst);
        System.out.println("analytics committed offsets: " + log.committedOffsets("analytics"));
    }

    private static void printBatch(String label, List<PartitionedLog.Message<String>> batch) {
        System.out.println(label);
        if (batch.isEmpty()) {
            System.out.println("  (empty)");
            return;
        }

        for (PartitionedLog.Message<String> message : batch) {
            System.out.println(
                "  p"
                    + message.getPartition()
                    + "@"
                    + message.getOffset()
                    + " key="
                    + message.getKey()
                    + " payload="
                    + message.getPayload()
            );
        }
    }
}
```